Journal of Systems Architecture 165 (2025) 103434

Contents lists available at ScienceDirect

Journal of Systems Architecture

journal homepage: www.elsevier.com/locate/sysarc

Contention-aware workflow scheduling on heterogeneous computing systems with shared buses

Yiming Zheng, Quanwang Wu *, Kun Cai, Yunni Xia

College of Computer Science, Chongqing University, Chongqing 401331, China

ARTICLE INFO

Keywords:
Clustering
Communication contention
Heterogeneous computing systems
Shared bus
Workflow scheduling

ABSTRACT

Heterogeneous computing systems (HCSs), which balance performance and efficiency by leveraging diverse computation resources, are widely used for executing workflow applications. These computation resources are typically interconnected through shared buses. When multiple participants simultaneously transmit data, communication contention arises, leading to longer communication time than expected. Nevertheless, the contention issue for the shared bus is rarely investigated in the literature. This paper proposes a Contention-Aware Clustering-based List scheduling (CACL) method to effectively address workflow scheduling in shared bus-based HCSs. In CACL, tasks are grouped into clusters based on criticality, clusters are mapped to computation resources, and then all tasks and edges in the workflow are scheduled to computation and communication resources in the HCS. Experimental results on realistic workflows demonstrate that CACL effectively addresses the communication contention issue, and reduces scheduling length by 5%–25% compared to existing methods, making it a robust solution for workflow scheduling in shared-bus-based HCSs.

1. Introduction

Heterogeneous computing systems (HCSs) integrate a diverse range of processing elements (PEs), e.g., CPUs, GPUs, and DSPs, to balance performance, energy efficiency, and scalability. These systems are widely employed in domains requiring efficient processing of diverse and complex tasks, such as high-performance computing, cloud computing, and machine learning [1]. The large-scale applications deployed in HCSs can contain hundreds or thousands of computing tasks. The workflow model is extensively applied to represent these applications through direct acyclic graphs (DAGs), where vertices denote tasks, and edges signify dependencies and data transfers among them.

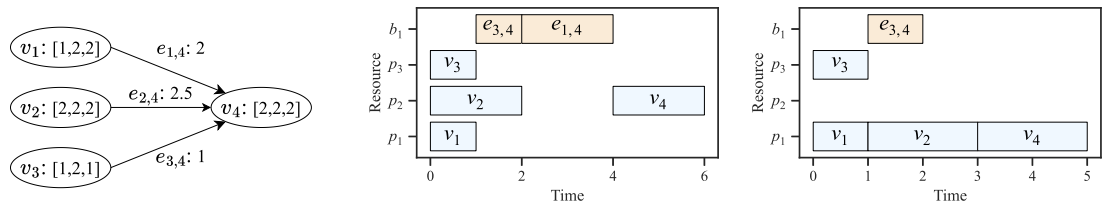
The shared bus-based communication architecture has been widely used in HCSs due to its cost-effectiveness and simplicity. It provides a unified communication medium that simplifies data transfer between PEs and memory subsystems. In embedded and heterogeneous systems, bus standards such as the Processor Local Bus typically support data transfer rates in the range of tens to several hundreds of megabytes per second, depending on bus width and clock frequency. When executing workflow applications in HCSs with shared buses, communication contention arises if multiple data transfers are carried out on the same bus simultaneously. Communication contention results in prolonged communication delays and degrades overall workflow performance [2]. The substantial communication volume and intricate structures in workflows further intensify this issue, making effective workflow scheduling a critical challenge [3,4].

Existing scheduling algorithms often overlook or inadequately address the impact of contention on communication performance. For instance, many traditional approaches assume idealized communication conditions, which leads to suboptimal performance when deployed on real-world shared bus-based systems [5]. While recent advancements, such as CC-TMS (Contention Cognizant Task and Message Scheduler) by Roy et al. [6], have improved in handling bus contention, they still face limitations in managing the dynamic nature of communication surges and varying workloads.

To illustrate the communication contention issue for workflow scheduling in shared bus-based HCSs, we present a simplified workflow scenario in Fig. 1(a) where the task computation time and edge transmission time are specified. For example, the computation time of task v_1 on PEs p_1, p_2 and p_3 is 1, 2, 2, respectively, and the transmission time of edge $e_{1,4}$ on bus b_1 is 2. When traditional classic workflow scheduling methods, such as HEFT (Heterogeneous Earliest Finish Time) [7], are applied to this workflow, the resulting solution is illustrated in Fig. 1(b). This visualization reveals a contention issue where $e_{3,4}$ and $e_{1,4}$ compete for the communication bus b_1: $e_{1,4}$ can begin

* Corresponding author.
E-mail addresses: 202214131100@stu.cqu.edu.cn (Y. Zheng), wqw@cqu.edu.cn (Q. Wu), 202214131123@cqu.edu.cn (K. Cai), xiayunni@hotmail.com (Y. Xia).

https://doi.org/10.1016/j.sysarc.2025.103434
Received 10 September 2024; Received in revised form 8 April 2025; Accepted 28 April 2025
Available online 16 May 2025
1383-7621/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.



(a) Workflow example (b) Solution by HEFT (c) Solution based on Clustering

Fig. 1. An illustration on communication contention for the shared bus.

transferring after $e_{3,4}$ finishes, leading to significant waiting time for all PEs. In contrast, grouping vertices 1, 2, and 4 into a cluster based on their prolonged data transmission time and allocating them to the same PE leads to a significantly improved solution, as illustrated in Fig. 1(c). This clustering strategy effectively mitigates communication contention, resulting in a notable reduction in the overall makespan.

Hence, this paper proposes a `Contention-Aware Clustering-based List Scheduling` (CACL) algorithm for workflow scheduling in shared bus-based HCSs. To mitigate the negative effects of communication contention, CACL groups tasks into clusters based on criticality, and allocates clusters to computation resources. Then, it schedules all tasks and edges in the workflow to computation and communication resources, respectively. Evaluation experiments are conducted using realistic workflows under diverse settings. The results show that CACL significantly enhances scheduling performance for workflows deployed on shared bus-based HCSs. In particular, CACL outperforms existing methods in over 95% of cases, reducing makespan by 5%–25%.

The remainder of this paper is organized as follows. Section 2 discusses related work. Section 3 outlines the scheduling model. Section 4 details the CACL algorithm. Section 5 presents the experimental design, results, and analysis. Finally, Section 6 concludes this paper and points out future work.

2. Related work

Traditionally, researchers investigate the workflow scheduling problem assuming that all PEs in distributed systems are fully connected. However, in some practical HCSs, PEs share communication resources (e.g., buses), which may lead to communication contention when multiple transmissions occur simultaneously. Below, we review the literature in two categories: workflow scheduling in fully connected networks assuming no communication contention, and contention-aware workflow scheduling for shared-resource networks.

2.1. Workflow scheduling in fully connected networks

In this category, PEs communicate through a fully connected network, indicating that there is a dedicated bidirectional communication channel for every pair of PEs. When multiple pairs of PEs conduct data transmission simultaneously, no contention arise. Researchers primarily focus on optimizing task allocation for better performance. Generally, three types of algorithms have been proposed to tackle the problem: list scheduling, duplication-based scheduling, and clustering scheduling.

List scheduling rely on a predefined priority list to schedule tasks, typically executed in two stages: (i) task prioritization, which assigns a priority to each task; and (ii) resource allocation, where tasks are sequentially assigned to PEs. Prominent list scheduling algorithms for workflow scheduling include HEFT proposed by Topcuoglu et al. [7], PEFT (Predict Earliest Finish Time) introduced by Arabnejad and Barbosa [8], and GRP-HEFT (Greedy Resource Provisioning and modified HEFT) developed by Faragardi et al. [9], all focusing on optimizing workflow execution time. For example, PEFT leverages an optimistic cost table (OCT) based on a look-ahead strategy to prioritize tasks, and schedules them to PEs one by one. In contrast to these single-objective algorithms, MOHEFT (Multi-Objective Heterogeneous Earliest Finish Time), proposed by Durillo and Prodan [10], optimizes both execution time and cost.

Duplication-based scheduling reduces communication delays by replicating tasks across multiple processors, allowing for earlier task initiation. For example, Ahmad and Kwok [11] introduced the Critical Path Fast Duplication (CPFD) algorithm to reduce communication overhead in message-passing systems by duplicating tasks across multiple processors. Bajaj and Agrawal [12] developed the Task Duplication-based Scheduling Algorithm for Network of Heterogeneous Systems (TANH), which provides improved scalability and performance over traditional algorithms.

Clustering algorithms generally involve two phases: tasks in a workflow are first divided into several disjoint sets, and then these sets are mapped to PEs. For instance, the Linear Clustering (LC) algorithm [13] reduces communication overhead by grouping tasks on the critical path into separate clusters. Similarly, Boeres et al. [14] introduced CHP (Clustering for Heterogeneous Processors), which combines replication-based heuristics for homogeneous systems with mapping strategies for heterogeneous systems. Kanemitsu et al. [15] proposed CMWSL (Clustering for Minimizing the Worst Schedule Length) to reduce scheduling length in scenarios involving a large number of PEs. Building upon LC, Hu et al. [16] proposed the PLC (Parallel Linear Clustering) algorithm, which improves upon LC by introducing a clustering threshold to prevent clusters from expanding indefinitely, and employing parallelization techniques to speed up the clustering process.

2.2. Contention-aware workflow scheduling

In many practical scenarios, the assumption of full connectivity does not hold. Sinnen and Sousa [17] highlighted the importance of accounting for contention in shared communication channels to produce accurate and efficient scheduling results. In [18], they proposed a scheduling model based on a communication topology graph with a single bus. This model maps task graph communications to links in the topology while accounting for both endpoint and network contention. Their approach was further extended in [19] to include arbitrary processor networks, and improved in [20] through the incorporation of task duplication. Choudhury et al. [21] proposed an online scheduling method for homogeneous multiprocessor embedded systems that handles dynamically changing task graphs due to conditional paths or task execution time variations.

More recently, Roy et al. [22] modeled contention effects in heterogeneous systems by formalizing bus architecture constraints and proposed an integer linear programming (ILP) method for workflows. Their ILP-based method, while theoretically rigorous, encounters scalability limitations with growing task numbers. To address this, they introduced CC-TMS [6], a contention-aware list scheduling algorithm that prioritizes tasks and simultaneously schedules communication edges, assigning tasks to processors for their earliest possible completion using a greedy strategy. However, CC-TMS underperforms when handling

Table 1
Key notations.

Category	Notation	Definition
Workflow	v_i	The ith computational task/vertex
	$e_{i,j}$	The edge from v_i to v_j
	$*\, n_i$	The set of v_i's immediate predecessors
	n_i^*	The set of v_i's immediate successors
	Γ_i	The set of v_i's adjacent edges
Platform	p_m	The mth PE
	b_n	The nth communication bus
	R_i^m	Running time of v_i on p_m
	$T_{i,j}^n$	Transmission time of $e_{i,j}$ on b_n
Schedule	$t_\circ(v_i, p_m)$	Execution start time of v_i on p_m
	$t_\bullet(v_i, p_m)$	Execution finish time of v_i on p_m
	$t_\circ(v_i, p_m)$	Earliest time p_m can execute v_i
	$t_\circ(e_{i,j}, b_n)$	Transmission start time of $e_{i,j}$ on b_n
	$t_\bullet(e_{i,j}, b_n)$	Transmission finish time of $e_{i,j}$ on b_n
	$t_\diamond(e_{i,j}, b_n)$	Earliest time b_n can transmit $e_{i,j}$

large volumes of communication data. Besides, Roy et al. proposed a slack aware frequency level allocator to co-schedule a set of periodic real-time applications in HCSs with shared buses [23].

For the HCSs provided by cloud computing, communication contention may occur during workflow execution. For example, when deploying workflows on hybrid clouds, cross-cloud data transmission can become a bottleneck. To address this, Zhang et al. [24] proposed a contention-cognizant list scheduling heuristic with task duplication to minimize workflow makespan. Similarly, Shu et al. [25] focused on executing workflows in geo-distributed datacenters, and proposed a scheduling method to effectively minimize makespan while preserving privacy and resolving communication contention.

Although some contention-aware workflow scheduling methods for HCSs have been proposed, they are unable to effectively handle communication contention when it becomes severe in shared-bas-based HCSs with irregular communication patterns. To advance the state of the art, this paper proposes an effective contention-aware clustering-based list scheduling method for this issue.

3. Workflow scheduling model on shared bus-based HCSs

This section first introduces workflow applications and shared-bus-based HCSs, followed by the scheduling model and formulation of the problem we address. The key notations used in this paper are shown in Table 1.

3.1. Workflow application

In HCSs, a complex application may comprise multiple interdependent computational tasks. The workflow model is widely used to represent such application through a Directed Acyclic Graph, DAG = (V, E). Here, a vertex $v_i \in V$ represents the ith computational task in the workflow, and a directed edge $e_{i,j} \in E$ indicates that v_i is the immediate predecessor of v_j and v_j can start execution only after v_i finishes. If $e_{i,j}$ is attached with data transmission, v_i can start only after the data from v_j has been received.

A computational task can have multiple predecessor and successor vertices. We use $*\, n_i$ and n_i^* to represent the sets of v_i's immediate predecessor and successor vertices, respectively. That is,

$$*\, n_i = \{v_x \in V \mid e_{x,i} \in E\},$$

$$n_i^* = \{v_x \in V \mid e_{i,x} \in E\}.$$

The set of v_i's adjacent edges, denoted as Γ_i, includes all incoming and outgoing edges connected to v_i, i.e.,

$$\Gamma_i = \{e_{x,i} \mid e_{x,i} \in E\} \bigcup \{e_{i,x} \mid e_{i,x} \in E\}.$$

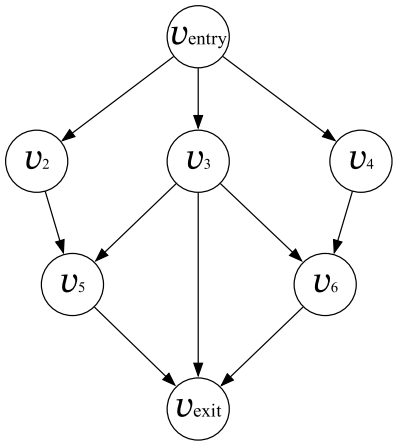


Fig. 2. An example of workflow application.

To generalize the workflow with one entry and one exit, two dummy tasks v_{entry} and v_{exit} with zero execution time are added to the beginning and end of the workflow, respectively. Hence, $*\, v_{entry} = \emptyset$, and $v_{exit}^* = \emptyset$. Fig. 2 illustrates a simple DAG, consisting of seven interdependent tasks.

3.2. Shared-bus-based HCS

The execution of workflow relies on the availability of computation resources and communication resources. Fig. 3 depicts the shared-bus-based HCS considered in this study. Specifically, let p_m be the mth computation resource (i.e., PE), and b_n be the nth communication bus. In the HCS, each PE can execute only one task at a time, and task execution is strictly non-preemptive. We use R_i^m to denote the running time of task v_i on PE p_m. Such shared-bus-based HCS considered are widely used in practice. For example, automotive control systems usually adopt an HCS with multiple shared buses (e.g., R-Car H3), enabling large-scale data transfers among processors and accelerator modules [26]. Although a communication bus can support multiple data transmissions simultaneously, the experimental results in [24,25] demonstrate that an exclusive communication mode outperforms the shared communication mode in workflow scheduling. Hence, we follow the former mode and use $T_{i,j}^n$ to represent the transmission time required for $e_{i,j}$ over bus b_n. Notably, when tasks v_i and v_j are allocated to the same PE, the transmission time becomes zero. Let $\bar{R}_i$ be the average runtime of v_i on all PEs P and $\bar{T}_{i,j}$ be the average transmission time of $e_{i,j}$ over all the buses B. The communication-to-computation ratio (CCR) of a workflow in a HCS is defined as,

$$\mathrm{CCR} = \frac{\sum_{e_{i,j} \in E} \bar{T}_{i,j}}{\sum_{v_i \in V} \bar{R}_i}. \tag{1}$$

3.3. Scheduling model

Assume that computational tasks v_i and v_j are executed on PEs p_l and p_m, respectively, and that there is an edge $e_{i,j}$ between them. If $p_l = p_m$, $e_{i,j}$ can start transmission immediately after v_i finishes. If $p_l \neq p_m$, it additionally requires that the used communication bus b_n is ready to transmit $e_{i,j}$. Let $t_\bullet(v_i, p_l)$ be the finish time of v_i on p_l, and $t_\diamond(e_{i,j}, b_n)$ denote the earliest ready time for b_n to transmit $e_{i,j}$. The transmission start time of $e_{i,j}$ can be determined via,

$$t_\circ(e_{i,j}, b_n) = \begin{cases} t_\bullet(v_i, p_l), & \text{if } p_l = p_m, \\ \max\{t_\bullet(v_i, p_l), t_\diamond(e_{i,j}, b_n)\}, & \text{if } p_l \neq p_m. \end{cases} \tag{2}$$

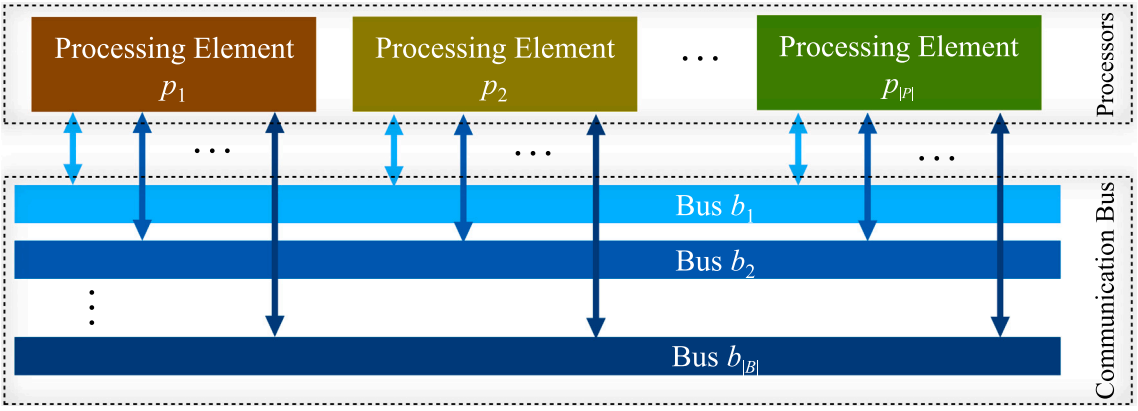


Fig. 3. The considered heterogeneous computing system.

The transmission finish time of $e_{i,j}$ can be calculated via,

$$t_{\bullet}(e_{i,j}, b_n) = t_{\circ}(e_{i,j}, b_n) + \begin{cases} 0, & \text{if } p_l = p_m, \\ T^n_{i,j}, & \text{if } p_l \neq p_m. \end{cases} \tag{3}$$

Task v_j can start execution only when all its input data has arrived at its assigned resource p_m and p_m is ready to execute it. Let $t_{\diamond}(v_j, p_m)$ represent the time p_m is ready to execute v_j, and $\beta_{k,j}$ be the bus where edge $e_{k,j}$ is transmitted. The start time and finish time of v_j on p_m can be calculated via:

$$t_{\circ}(v_j, p_m) = \max\left\{ t_{\diamond}(v_j, p_m), \max_{v_k \in *n_j} \{t_{\bullet}(e_{k,j}, \beta_{k,j})\} \right\}, \tag{4}$$

$$t_{\bullet}(v_j, p_m) = t_{\circ}(v_j, p_m) + R^m_j. \tag{5}$$

Since v_{exit} is the last task to be executed in a workflow, the workflow makespan is obtained as,

$$\text{makespan} = t_{\bullet}(v_{exit}, \rho_{exit}), \tag{6}$$

where, ρ_{exit} is the PE where v_{exit} is allocated.

Our goal is to schedule tasks and edges in a workflow onto PEs and buses in the HCS to minimize the makespan.

4. Contention-aware clustering-based list scheduling

In practical workflows, there are usually certain vertices with multiple incoming or outgoing edges. Around these vertices, the demand for communication resources increases sharply and the communication contention can emerge easily. Traditional list scheduling methods allocate resources using greedy strategies, such as selecting the earliest finish time. However, this approach often increases the likelihood of vertices competing for shared buses in HCSs. Such severe communication contention can lead to prolonged idleness of PEs, ultimately degrading scheduling performance. Traditional clustering scheduling methods can reduce communication overhead to some extent. However, they typically assume the availability of infinitely many homogeneous processors, overlooking resource constraints and hardware heterogeneity in HCSs. Furthermore, these methods often perform resource allocation at the cluster level, which makes it challenging to align with the fine-grained parallelism required at the vertex level.

Communication contention typically arises at vertices with many edges in HCSs with shared buses. Motivated by this observation, we aim to group vertices with significant data transfer requirements into clusters and allocate them to the same PEs, effectively eliminating communication overhead for these tasks. The remaining tasks are then scheduled alongside these clusters based on a priority list. In essence, our approach CACL integrates clustering into list scheduling to mitigate communication contention and enhance overall performance.

Below, we first outline the process of task clustering and the allocation of clusters to PEs in CACL. We then detail how tasks and edges are scheduled to PEs and buses, respectively. Finally, we analyze the computational complexity of CACL and present an illustrative example.

Algorithm 1: Criticality-Based Task Clustering.

Input: DAG=(V, E)
Output: Set of clusters $\mathbb{S}$
determine the set of critical vertices C via Eq. (7)
foreach $v_i \in V$ **do**
  $S_i \leftarrow -1$
Sort C in descending order of $|\Gamma_i|$
foreach $v_i \in C$ **do**
  $S_i \leftarrow i$
  $\theta \leftarrow$ calculated by Eq. (8)
  clustering(i, θ)
Construct clusters $\mathbb{S}$ based on the S value of all vertices;
return $\mathbb{S}$
Function `clustering`(i, θ):
  foreach *neighbor* $v_j \in * n_i \cup n_i^*$ **do**
    if $S_j \neq 0$ **then**
      continue
    if the average transmission time between v_i and $v_j > \theta$ **then**
      $S_j \leftarrow i$
      clustering(j, θ)

4.1. Criticality-based task clustering

To achieve a balance between reducing communication contention and maintaining parallelism, the Criticality-Based Task Clustering algorithm is proposed, as shown in Algorithm 1. Its core idea is to select a set of critical vertices based on certain criteria, and expand from them towards both their predecessors and successors, thereby increasing the number of vertices in their clusters.

The set of critical vertices C is determined based on the graph properties of $G = (V, E)$ (e.g., its CCR and edge density $\frac{|E|}{|V|}$) and the HCS platform (e.g., $|B|$ and $|P|$) as follows,

$$C = \left\{ v_i \in V \;\middle|\; |\Gamma_i| \geq \alpha \left(\frac{|E|}{|V|} + \frac{|B|}{|P| \times \text{CCR}} \right) \right\}. \tag{7}$$

Specifically, a vertex v_i is critical when its neighborhood connectivity $|\Gamma_i|$ in the graph is no less than a specific threshold. The threshold is established adaptively by combining the edge density $\frac{|E|}{|V|}$, and the resource balance factor $\frac{|B|}{|P| \times \text{CCR}}$, which is larger when there are more buses or fewer processors, or the CCR value is less. The scaling factor α

is additionally introduce to adjust the strictness of the selection criteria: increasing α makes the selection stricter, while decreasing α relaxes the condition. By default, α is set to 1.

Let $\bar{T}_{i,j}$ be the average transmission time of the edge $e_{i,j}$ on all buses. The threshold θ_i for adding other vertices to c_i's cluster is set to the average transmission time from c_i to all vertices in Γ_i as follows:

$$\theta_i = \frac{1}{|\Gamma_i|}\left(\sum_{v_j \in *n_i} \bar{T}_{j,i} + \sum_{v_j \in n_i^*} \bar{T}_{i,j}\right). \tag{8}$$

In Algorithm 1, the set of critical vertices C are first determined according to Eq. (7) (Line 1). Then, the cluster index S_i for each vertex is set to 0, indicating that it does not belong to any cluster (Lines 2–3). All critical vertices in C are sorted in descending order of $|\Gamma_i|$ (Line 4). For each critical vertex c_i in C, it is put into the ith cluster (Line 6) and the threshold θ_i for adding other vertices to this cluster is calculated using Eq. (8) (Line 7). The `clustering` function, to be explained later, is invoked to recursively add vertices to the cluster (Line 8). After the iteration, clusters $\mathbb{S}$ are constructed by grouping vertices with the same S value, not equal to -1, into a cluster (Line 9).

In the `clustering` function, all the adjacent vertices of v_i are traversed (Line 12). If the adjacent vertex v_j has already been assigned to a cluster (i.e., $S_j \neq 0$), it is skipped (Lines 13–14). It is put into the cluster S_i if the average transmission time between v_i and v_j exceeds the threshold θ (Lines 15–16). Besides, the `clustering` function is recurred with the input of j and θ.

4.2. Cluster scheduling

Here, the rank properties of tasks and clusters are determined, and the task clusters are allocated to PEs sequentially based on their ranks by using list scheduling.

4.2.1. Rank determination

The Optimistic Cost Table (OCT) from [8] measure the maximum value of the shortest path from a vertex to the exit vertex. For vertex v_i on PE p_m, the OCT value can be defined by traversing backwards from v_{exit} to v_{entry} as follows:

$$\mathrm{OCT}(v_i, p_m, b_n) = \max_{v_j \in n_i^*} \left[\min_{p_x \in P, b_y \in B} \{\mathrm{OCT}(v_j, p_x, b_y) + R_j^x + T_{i,j}^n\}\right], \tag{9}$$

where, the transmission time $T_{i,j}^n$ of $e_{i,j}$ on bus b_n is 0 if $p_m = p_x$, and $T_{i,j}^n$ otherwise. $\mathrm{OCT}(v_i, p_m, b_n)$ represents the optimistic execution time for v_i's child vertices, considering both computational and communication resources that minimize the execution time.

The rank of vertex v_i is determined by averaging its OCT values across all PEs and buses, i.e.,

$$\mathrm{rank}(v_i) = \frac{1}{|P| \cdot |B|} \sum_{p_m \in P, b_n \in B} \mathrm{OCT}(v_i, p_m, b_n). \tag{10}$$

Similarly, the rank of a task cluster S_i is defined as,

$$\mathrm{rank}(S_i) = \frac{1}{|S_i|} \sum_{v_j \in S_i} \mathrm{rank}(v_j). \tag{11}$$

4.2.2. Allocating task clusters to PEs

Algorithm 2 outlines how task clusters are allocated to PEs. First, the rank of each cluster is calculated using Eq. (11), and the clusters are sorted in descending order of cluster ranks (Lines 1–2). The new ready time Ω_m of each processor p_m is initialized to 0 (Lines 3–4). For each cluster S_i, the earliest completion time $t_\bullet(S_i, p_m)$ on each PE is estimated based on resource availability (Lines 6–8) via,

$$t_\bullet(S_i, p_m) = \Omega_m + \sum_{v_j \in S_i} R_j^m. \tag{12}$$

The cluster is then allocated to the PE that minimizes $t_\bullet(S_i, p_m)$, and the new ready time of the selected PE is updated accordingly (lines 8-9).

Algorithm 2: Task Cluster Scheduling

Input: Set of clusters $\mathbb{S}$
Output: cluster and PE mapping $\mathcal{A} : \mathbb{S} \to P$

1 Compute the rank of each cluster using Eq. (11)
2 Sort $\mathbb{S}$ in descending order of cluster ranks
3 **foreach** *PE* $p_m \in P$ **do**
4 $\Omega_m \leftarrow 0$
5 Initialize allocation mapping $\mathcal{A} \leftarrow \emptyset$
6 **foreach** *cluster* $S_i \in \mathbb{S}$ **do**
7 **foreach** *PE* $p_m \in P$ **do**
8 Calculate $t_\bullet(S_i, p_m)$ using Eq. (12)
9 Let p_{best} be the PE that minimizes $t_\bullet(S_i, p_m)$
10 $\mathcal{A}[S_i] \leftarrow p_{best}$
11 $\Omega_* \leftarrow t_\bullet(S_i, p_{best})$
12 **return** $\mathcal{A}$

4.3. Task and edge scheduling

In this subsection, all the vertices and edges in the workflow are scheduled to specific time slots of the communication and computation resources through list scheduling.

To capture the effect of tasks that will be allocated to the current PE based on cluster scheduling, we introduce τ_m to represent the reserved time for PE p_m. Let S_i' represent the set of vertices in cluster S_i that have not yet been scheduled. If the cluster S_i is scheduled to PE p_m, the reserved time $\tau_{m,i}$ for p_m with respect to S_i' is calculated as,

$$\tau_{m,i} = \sum_{v_j \in S_i'} R_j^m. \tag{13}$$

For p_m, the total reserved time τ_m is,

$$\tau_m = \sum_{S_i \in \mathbb{S}} \tau_{m,i}. \tag{14}$$

Using τ_m, we introduce the virtual earliest finish time $\tilde{t}_\bullet(v_i, p_m)$ to guide the scheduling process. It is calculated as,

$$\tilde{t}_\bullet(v_i, p_m) = t_\bullet(v_i, p_m) + \tau_m. \tag{15}$$

Algorithm 3: Schedule with Virtual Earliest Finish Time (VEFT)

Input: DAG=(V, E), cluster and PE mapping $\mathcal{A}$

1 $\mathcal{L} \leftarrow$ sort tasks in V in non-increasing order of *rank*
2 **foreach** vertex $v_i \in \mathcal{L}$ **do**
3 $*n_i' \leftarrow$ sort tasks in $*n_i$ in descending order of $\bar{T}_{x,i}$
4 $p_{best} \leftarrow$ obtain the PE where v_i is allocated from $\mathcal{A}$
5 **if** $p_{best} \neq NULL$ **then**
6 update τ_{best} using Eq. (14)
7 **else**
8 **foreach** processor $p_m \in P$ **do**
9 Tentatively assign v_i to p_m
10 **foreach** predecessor $v_x \in *n_i'$ **do**
11 **foreach** $b_n \in B$ **do**
12 Compute $t_\bullet(e_{x,i}, b_n)$ by Eqs. (2), (3)
13 Tentatively assign $e_{x,i}$ to the bus b_n that minimizes $t_\bullet(e_{x,i}, b_n)$
14 withdraw the above tentative operations
15 Compute the virtual earliest finish time $\tilde{t}_\bullet(v_i, p_m)$
16 $p_{best} \leftarrow$ the PE that minimizes $\tilde{t}_\bullet(v_i, p_m)$
17 schedule v_i's incoming edges to buses that minimize their finish time by Eqs. (2), (3)
18 schedule v_i to p_{best} by Eqs. (4), (5)

The overall scheduling process based on the virtual earliest finish time $\tilde{t}_\bullet$ is detailed in Algorithm 3. Initially, a task list $\mathcal{L}$ in descending

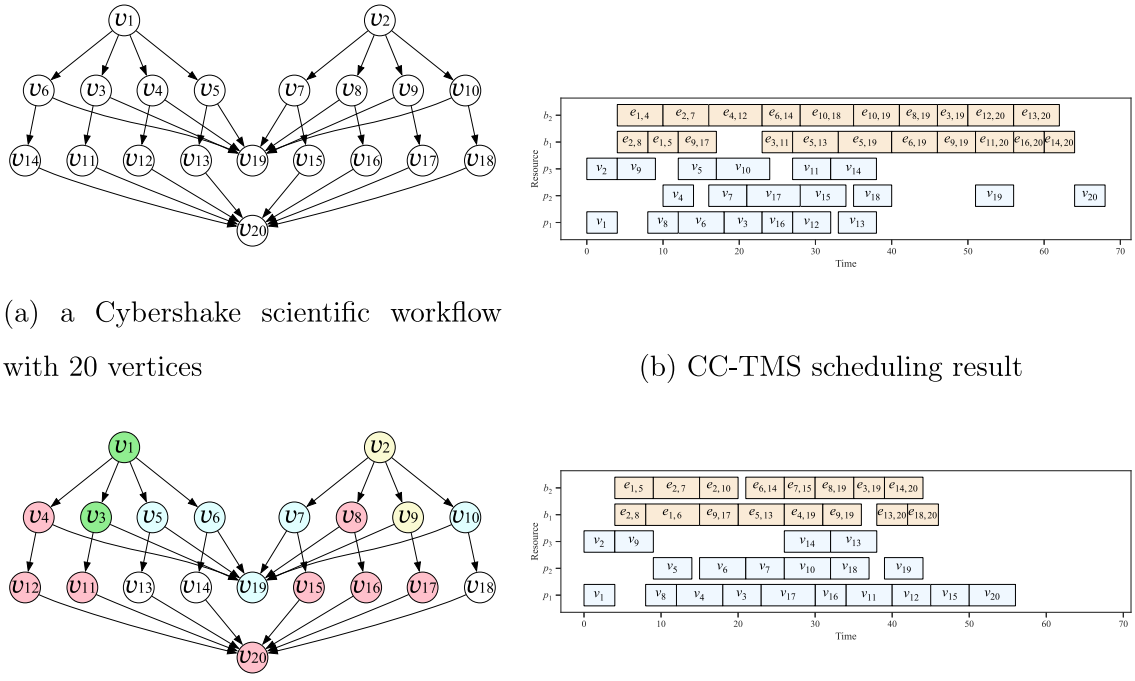


Fig. 4. Illustrative examples.

order of task priority (*rank*) is constructed (Line 1). Then, each task in $\mathcal{L}$ is traversed and scheduled with its incoming edges. For each task v_i, all its immediate predecessors are sorted in descending order of the average transmission time between it and v_i, i.e., $\bar{T}_{x,i}$ (Line 3). If v_i has already been allocated to a PE, the reserved time for the PE is updated based on Eq. (14) (Lines 3–5).

Otherwise, the algorithm tentatively assigns v_i to each processor $p_r \in P$. For each tentative assignment, it also assigns each incoming edge $e_{x,i}$ to bus $b_k \in B$ that minimize the finish time $t_\bullet(e_{x,i})$. Based on these tentative assignments, the virtual earliest finish time $\tilde{t}_\bullet(v_i, p_m)$ is obtained for all processors. The PE that minimizes $\tilde{t}_\bullet(v_i, p_m)$ is selected for v_i (Lines 16).

Afterwards, v_i and its incoming edges are all scheduled to specific time slots of the communication and computation resources (Lines 17–18).

4.4. Complexity analysis

In a workflow with n tasks and m edges, CACL first traverses all tasks and edges in the workflow graph to identify the set of critical vertices C, with a time complexity of $O(n)$. Let k represent the number of critical vertices identified. The recursive clustering function then processes all predecessors and successors of each vertex, visiting all vertices and edges, leading to a time complexity of $O(n + m)$. Therefore, the total time complexity of Algorithm 1 is: $O(n) + k \cdot O(n + m) = O(kn + km)$. In Algorithm 2, computing the OCT requires $O(|P| \cdot |B| \cdot (n+m))$ and cluster scheduling for $\mathbb{S}$ has a complexity of $O(k \cdot |P|)$. For the task scheduling phase in Algorithm 3, its worst-case complexity is $O(n^2 \cdot |P| \cdot |B|)$. Thus, the overall time complexity is: $O(kn + km) + O(|P| \cdot |B| \cdot (n + m)) + O(k \cdot |P|) + O(n^2 \cdot |P| \cdot |B|)$. Since k, $|P|$, and $|B|$ are typically much smaller than n and m, the practical time complexity simplifies to: $O(n^2 \cdot |P| \cdot |B|)$.

4.5. Illustrative example

Fig. 4(a) presents a Cybershake scientific workflow consisting of 20 vertices in a HCS equipped with 3 PEs and 2 buses. The corresponding task computation time and edge transmission time are listed in Table 2, where the data generation approach follows the description in [6]. It is scheduled using both the CC-TMS and CACL algorithms. The scheduling schemes produced by them are depicted in Fig. 4(b) and 4(d), respectively, and their makespan values are 68 and 56.

Table 2
Task computation time and edge transmission time for Fig. 4(a).

Vertex	p_1	p_2	p_3	Edge	b_1	b_2	Edge	b_1	b_2
v_1	4	7	6	$e_{1,3}$	4	6	$e_{7,19}$	5	7
v_2	4	5	4	$e_{1,4}$	5	6	$e_{8,19}$	7	5
v_3	5	4	7	$e_{1,5}$	4	5	$e_{9,19}$	5	6
v_4	6	4	7	$e_{1,6}$	7	6	$e_{10,19}$	7	6
v_5	6	5	5	$e_{2,7}$	6	6	$e_{11,20}$	5	7
v_6	6	6	6	$e_{2,8}$	4	4	$e_{12,20}$	6	6
v_7	7	5	5	$e_{2,9}$	5	7	$e_{13,20}$	4	6
v_8	4	5	6	$e_{2,10}$	7	5	$e_{14,20}$	4	5
v_9	4	6	5	$e_{3,11}$	4	4	$e_{15,20}$	5	6
v_{10}	6	6	7	$e_{4,12}$	4	7	$e_{16,20}$	4	7
v_{11}	6	5	5	$e_{5,13}$	6	7	$e_{17,20}$	5	7
v_{12}	5	6	4	$e_{6,14}$	6	5	$e_{18,20}$	4	5
v_{13}	5	5	6	$e_{7,15}$	6	4			
v_{14}	7	5	6	$e_{8,16}$	6	7			
v_{15}	5	6	5	$e_{9,17}$	5	5			
v_{16}	4	7	7	$e_{10,18}$	5	7			
v_{17}	7	7	7	$e_{3,19}$	6	4			
v_{18}	4	5	4	$e_{4,19}$	5	7			
v_{19}	7	5	7	$e_{5,19}$	7	6			
v_{20}	6	4	7	$e_{6,19}$	6	6			

As shown in Fig. 4(b), by using CC-TMS, there is severe communication contention for vertices such as v_{19} and v_{20} with multiple incoming edges, leading to great PE idle time and a larger workflow makespan value. In contrast, Fig. 4(d) demonstrates that CACL tackles communication contention more effectively than CC-TMS by combining list scheduling and cluster clustering. Specifically, the resulting clusters are shown in Fig. 4(c), where different colors represent different clusters. Some vertices remain un-clustered due to threshold settings and are displayed in white.

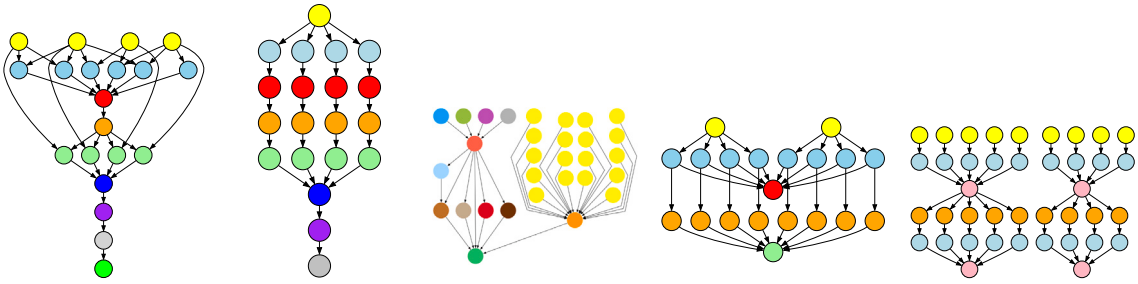


Fig. 5. The structure of five scientific workflows.

5. Experimental evaluation

In this section, we evaluate the performance of the proposed algorithm using real-world scientific workflows. The experiments were simulated on a machine with 16 GB of RAM, an Intel Core i7 processor, running Windows 11, and using the Java 17 platform.

5.1. Experimental settings

The Pegasus project from the University of Southern California has released many realistic scientific workflow applications, which are widely used to evaluate scheduling algorithms [27,28]. In this study, we conducted experiments on five of the most popular real workflows: Cybershake, Genome, Ligo, Montage, and Sipht. The structures of these workflows differ from each other, and their relatively small-scale instances are shown in Fig. 5. We chose ten sizes of each workflow type for experiments, i.e., the number of vertices $|V|$ = [100, 200, 300, 400, 500, 600, 700, 800, 900, 1000].

For HCS, we considered five different settings for computation and communication resources: the PE count $|P|$ = [3, 6, 9, 12, 15] and the bus count $|B|$ = [1, 2, 3, 4, 5]. For communication-to-computation ratio (CCR), we used 7 settings: CCR = [0.5, 1, 1.5, 2, 2.5, 3, 3.5]. Based on the chosen CCR, the task runtime on each PE and edge transmission time on each bus were randomly generated.

The following three methods are used for comparison: CC-TMS [6], PLC [16], and PEFT [8]. CC-TMS is specifically designed for bus-based HCSs. PLC is a clustering-based scheduling method and PEFT is a classical list scheduling method. Since PLC and PEFT cannot deal with communication contention, they are adapted by incorporating edge scheduling techniques from Algorithm 3 to account for bus contention, ensuring a fair comparison with our proposed algorithm.

The metrics used in the experiments include speedup and the proportion of better solutions.

(1) **Speedup:** The makespan is the most commonly used metric to evaluate scheduling quality. Given the substantial differences in the topology of real workflows, we use the normalized makespan, referred to as $Speedup$, to ensure more interpretable results. For a given DAG, speedup is defined as follows:

$$Speedup = \frac{\min_{p_m \in P} \left\{ \sum_{v_i \in V} R_i^m \right\}}{\text{makespan}(solution)}, \tag{16}$$

where, makespan($solution$) refers to the makespan of the yielded scheduling solution. It represents the ratio of sequential execution time to parallel execution time, with higher values indicating better algorithm performance.

(2) **Proportion of Better Solutions:** We compare the performance of the proposed algorithm CACL against the baselines across different settings. We calculate the proportion of cases where CACL produces better, worse, or equal results than a compared algorithm. These results are summarized in a pairwise comparison table.

Table 3
Pairwise comparison with different task numbers.

		PLC	CC-TMS	PEFT	CACL
PLC	better	*	72.3%	49.6%	0.3%
	equal		1.1%	1.5%	0.0%
	worse		26.6%	48.9%	99.7%
CC-TMS	better	26.6%	*	70.4%	0.2%
	equal	1.1%		0.5%	0.0%
	worse	72.3%		29.1%	99.8%
PEFT	better	48.9%	29.1%	*	0.0%
	equal	1.5%	0.5%		0.1%
	worse	49.6%	70.4%		99.9%
CACL	better	99.7%	99.8%	99.9%	*
	equal	0.0%	0.0%	0.1%	
	worse	0.3%	0.2%	0.0%	

5.2. Experimental results

We conducted experiments on five scientific workflows, varying one experimental variable at a time—specifically, the vertex count $|V|$ in a workflow, CCR, PE count $|P|$, and bus count $|B|$.

5.2.1. Impact of $|V|$

In this experiment, the vertex count is increased from 100 to 1000 at the step of 100, and the other three parameters are fixed as follows: CCR = 1, $|P|$ = 3, $|B|$ = 2. For each set of conditions, such as {$|V|$ = 100, CCR = 1, $|P|$ = 3, $|B|$ = 2}, we generated 20 different workflows for each time and averaged their results to calculate the corresponding $Speedup$ value. The experimental results are shown in Fig. 6 and Table 3.

As shown in Fig. 6, the CACL algorithm consistently achieves higher speedup across different workflows, with its superiority becoming more pronounced as $|V|$ increases. This trend highlights CACL's scalability, particularly in large workflows where communication overhead and resource contention typically slow down traditional scheduling algorithms. By clustering tasks and optimizing resource allocation, CACL effectively reduces PE idle times and communication delays, leading to better performance as workflows grow in scale. In contrast, the compared methods struggle with larger $|V|$ values, suffering from inefficiencies due to communication contention. The results demonstrate CACL's strength in handling large-scale, communication-intensive workflows.

From Table 3, it is clear that CACL outperforms the other three algorithms, surpassing PLC in 99.7% of cases, CC-TMS in 99.8% of cases and PEFT in 99.9% of cases. In contrast, PEFT performs the worst, indicating that the clustering approach plays a crucial role in optimizing scheduling, especially when communication resources are limited.

Additionally, the average runtime for each algorithm to yield a solution for all the workflows is recorded and depicted in Fig. 7. We can observe that for each algorithm, the runtime generally increases with the number of vertices ($|V|$). The clustering-based algorithms (i.e., PLC

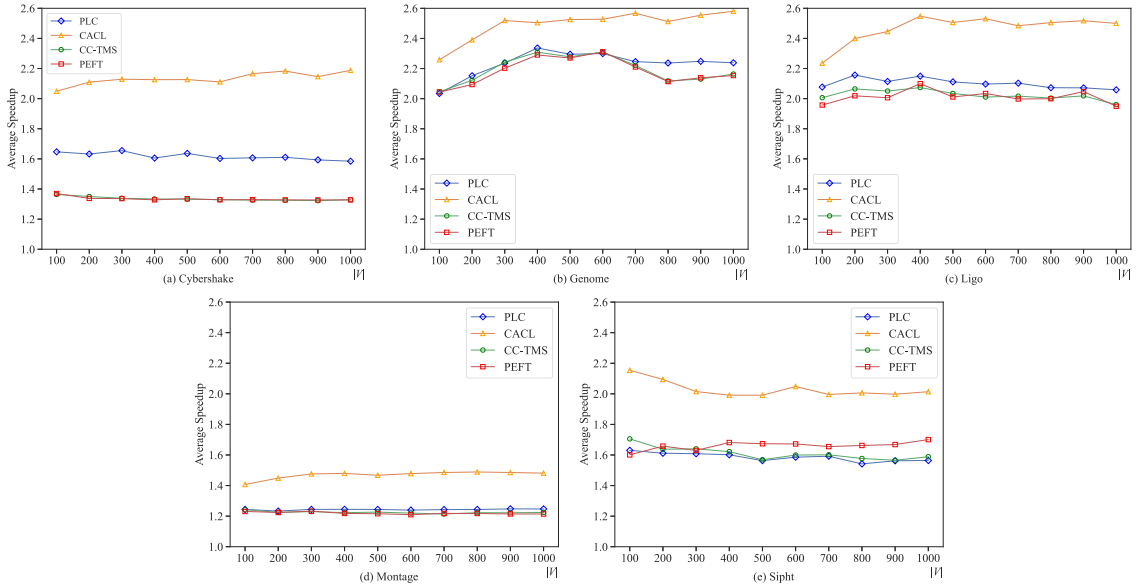


Fig. 6. Average speedup with different task numbers.

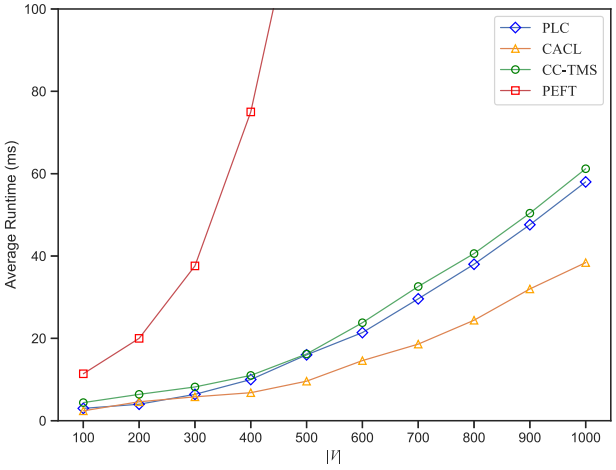


Fig. 7. Average runtime (ms) of each method with different task numbers.

and CACL) requires less runtime than PEFT and CC-TMS as tasks are first grouped. CACL consumes around 40 ms even when $|V| = 1000$. The low runtime overhead of CACL indicates that it can be applied to both static and dynamic workflow scheduling scenarios.

5.2.2. Impact of CCR

In this experiment, the impact of CCR on scheduling performance is evaluated. CCR of workflows was increased from 0.5 to 3.5 at the step of 0.5, while other variables were fixed as follows: $|V|$=200, $|P|$=6, $|B|$=2. For each setting, 20 workflow instances of each type were used. The experimental results are shown in Fig. 8 and Table 4.

As shown in Fig. 8, as CCR increases, performance declines for all algorithms. This is because a larger CCR value indicates higher data communication amount in the workflow. When the CCR value is small, traditional algorithms can also perform well and may even yield better results. For example, the PLC method performs the best when CCR is 0.5 and the workflow type is Ligo. Nevertheless, our method outperforms the compared methods in most cases. Specifically, Table 4 demonstrate the comparison results between CACL and its peers. CACL outperforms in the majority of cases: winning 81.8% of comparisons against PLC, 92.2% against CC-TMS and 94.7% against PEFT.

Table 4
Pairwise comparison under different CCR.

		PLC	CC-TMS	PEFT	CACL
PLC	better	*	81.1%	78.1%	18.1%
	equal		0.0%	0.1%	0.1%
	worse		18.9%	21.7%	81.8%
CC-TMS	better	18.9%	*	88.7%	7.8%
	equal	0.0%		0.0%	0.0%
	worse	81.1%		11.3%	92.1%
PEFT	better	21.7%	11.3%	*	5.3%
	equal	0.1%	0.0%		0.0%
	worse	78.1%	88.7%		94.7%
CACL	better	81.8%	92.2%	94.7%	*
	equal	0.1%	0.0%	0.0%	
	worse	18.1%	7.8%	5.3%	

Table 5
Pairwise comparison under different number of PEs.

		PLC	CC-TMS	PEFT	CACL
PLC	better	*	77.8%	79.6%	5.4%
	equal		0.0%	0.0%	0.0%
	worse		22.2%	20.4%	94.6%
CC-TMS	better	22.2%	*	87.2%	0.4%
	equal	0.0%		0.0%	0.0%
	worse	77.8%		12.8%	99.6%
PEFT	better	20.4%	12.8%	*	0.0%
	equal	0.0%	0.0%		0.0%
	worse	79.6%	87.2%		100.0%
CACL	better	94.6%	99.6%	100.0%	*
	equal	0.0%	0.0%	0.0%	
	worse	5.4%	0.4%	0.0%	

5.2.3. Impact of $|P|$

This section presents the experimental results with the number of processors as the variable, ranging from [3,6,9,12,15], while other parameters were kept constant as follows: $|V|$=200, CCR=1, $|B|$=2. The experimental results are shown in Fig. 9 and Table 5.

As illustrated in Fig. 9, CACL consistently delivers better scheduling solutions across all PE counts. When increasing to a certain PE count, the speedup growth rate plateaus across all workflow types. Moreover, the overall trend of changes for all algorithms is relatively consistent. The results in Table 5 demonstrate CACL's dominance over the other

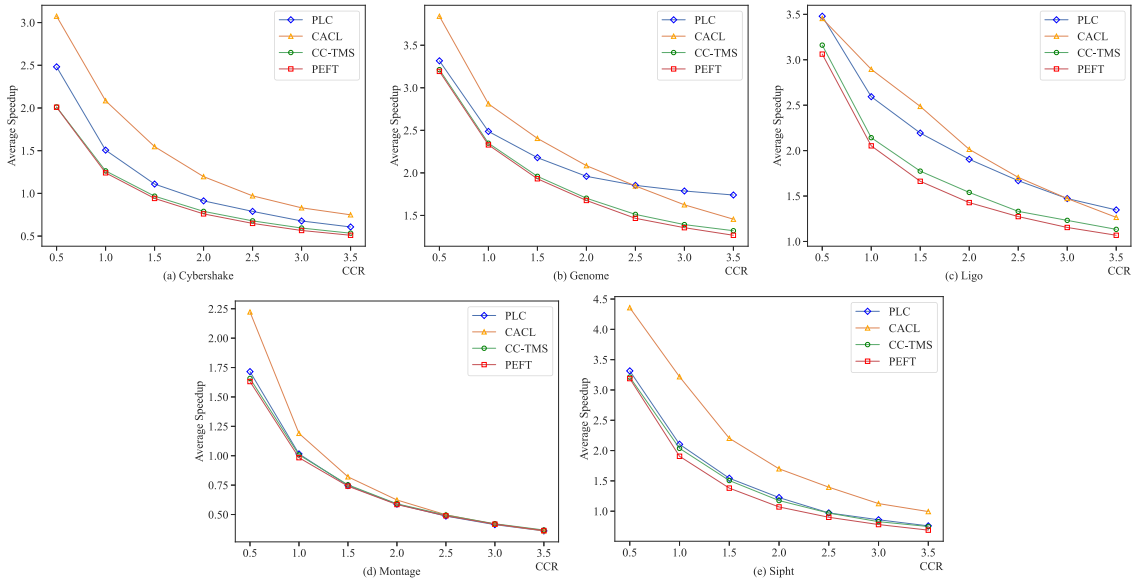


Fig. 8. Average speedup under different CCR.

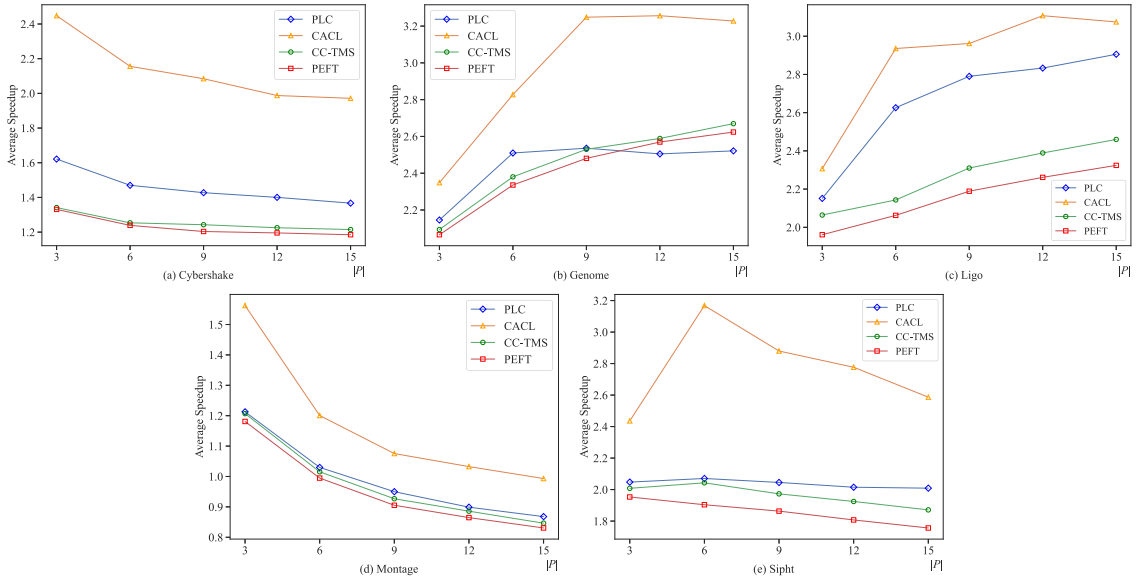


Fig. 9. Average speedup under different number of processors.

algorithms when varying the number of processors. CACL outperforms PLC in 94.6% of cases and TMS in 99.6%.

5.2.4. Impact of $|B|$

This section examines the impact of varying the number of buses, ranging from [1,2,3,4,5], and the other variables are held constant as follows, $|V|$=200, CCR=1, $|P|$=6. For each setting, we generated 20 workflow instances of each type for performance comparison. The experimental results are shown in Fig. 10 and Table 6.

As shown in Fig. 10, in most cases, CACL outperforms compared algorithms. However, for workflows Genome and Ligo, when $|B|$=5, CACL's performance is worse than compared methods. This is because when the communication resources are adequate, communication contention seldom occurs for these workflows, and traditional contention-oblivious scheduling algorithms can work well. By contrast, for workflows Montage and Sipht, the communication resource is insufficient even when $|B|$=5, and CACL can perform much better. The

Table 6
Pairwise comparison under different number of buses.

		PLC	TMS	PEFT	CACL
PLC	better	*	80.0%	78.2%	19.6%
	equal		0.2%	0.2%	0.0%
	worse		19.8%	21.6%	80.4%
TMS	better	19.8%	*	88.2%	13.2%
	equal	0.2%		0.0%	0.2%
	worse	80.0%		11.8%	86.6%
PEFT	better	21.6%	11.8%	*	10.6%
	equal	0.2%	0.0%		0.0%
	worse	78.2%	88.2%		89.4%
CACL	better	80.4%	86.6%	89.4%	*
	equal	0.0%	0.2%	0.0%	
	worse	19.6%	13.2%	10.6%	

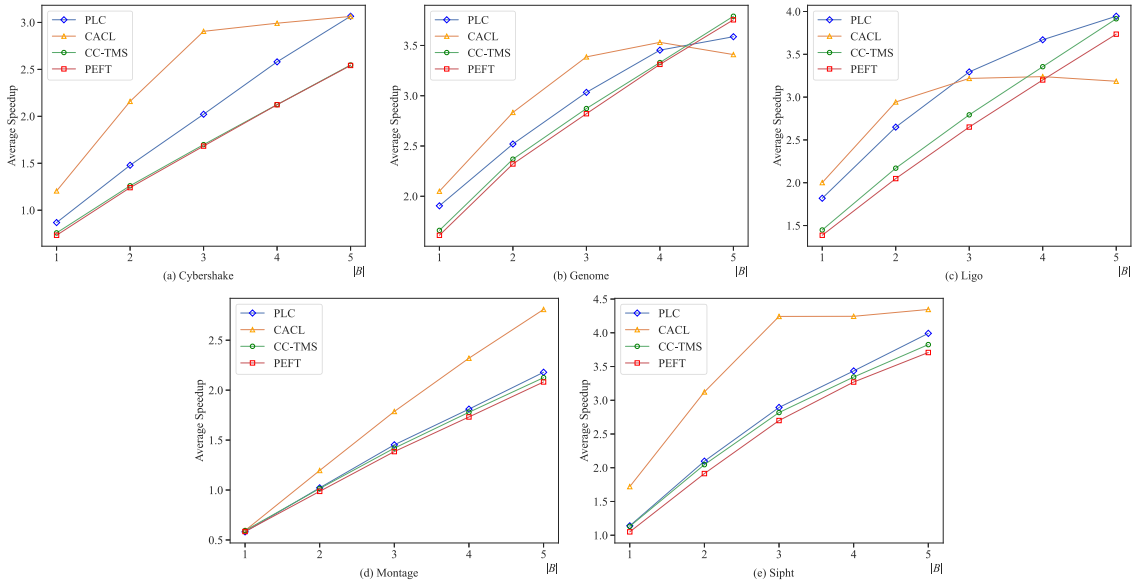


Fig. 10. Average speedup under different number of buses.

results indicate that CACL is highly efficient in scheduling workflows in a HCS in which communication contention may frequently occur.

6. Conclusion

In this study, we proposed a contention-aware clustering-based list scheduling (CACL) algorithm to schedule workflows in bus-based HCSs. CACL clusters tasks with heavy data communication amount and assigns them to the same PE. By conducting clustering-based list scheduling, CACL effectively reduces communication contention on shared buses and improves the allocation of both communication and computation resources. Extensive experiments on real-world scientific workflows were conducted. The results demonstrated that CACL performs much better than existing algorithms, particularly in the case of large-scale workflows, and limited communication resources. For future work, we are interested in extending CACL to other communication models and enhancing its adaptability to dynamic and resource-rich computing environments.

CRediT authorship contribution statement

Yiming Zheng: Writing – original draft, Visualization, Software. **Quanwang Wu:** Writing – review & editing, Validation, Supervision, Project administration, Methodology, Investigation, Formal analysis, Conceptualization. **Kun Cai:** Data curation. **Yunni Xia:** Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported in part by the National Natural Science Foundation of China (62172065, U22A2026, 62322601).

The authors would like to thank the editor and anonymous reviewers for their valuable comments/suggestions that help us improve this paper.

Data availability

Data will be made available on request.

References

[1] Z. Wang, H. Wang, X. Song, J. Wu, Communication-aware energy consumption model in heterogeneous computing systems, Comput. J. 67 (1) (2024) 78–94, http://dx.doi.org/10.1093/comjnl/bxac159.

[2] J. Arora, C. Maia, S.A. Rashid, G. Nelissen, E. Tovar, Bus-contention aware WCRT analysis for the 3-phase task model considering a work-conserving bus arbitration scheme, J. Syst. Archit. 122 (2022) 102345, http://dx.doi.org/10.1016/j.sysarc.2021.102345.

[3] K. Cai, Q. Wu, M. Zhou, C. Chen, J. Wen, S. Wang, Dynamically scheduling deadline-constrained interleaved workflows on heterogeneous computing systems, IEEE Trans. Serv. Comput. (2025) 1–12, http://dx.doi.org/10.1109/TSC.2025.3536317.

[4] J.M. Aceituno, A. Guasque, P. Balbastre, J. Simó, A. Crespo, Hardware resources contention-aware scheduling of hard real-time multiprocessor systems, J. Syst. Archit. 118 (2021) 102223, http://dx.doi.org/10.1016/j.sysarc.2021.102223.

[5] S. Blagodurov, S. Zhuravlev, A. Fedorova, Contention-aware scheduling on multicore systems, ACM Trans. Comput. Syst. (TOCS) 28 (4) (2010) http://dx.doi.org/10.1145/1880018.1880019.

[6] S.K. Roy, R. Devaraj, A. Sarkar, Contention cognizant scheduling of task graphs on shared bus-based heterogeneous platforms, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 41 (2) (2022) 281–293, http://dx.doi.org/10.1109/TCAD.2021.3059569.

[7] H. Topcuoglu, S. Hariri, Min-You Wu, Performance-effective and low-complexity task scheduling for heterogeneous computing, IEEE Trans. Parallel Distrib. Syst. 13 (3) (2002) 260–274, http://dx.doi.org/10.1109/71.993206.

[8] H. Arabnejad, J.G. Barbosa, List scheduling algorithm for heterogeneous systems by an optimistic cost table, IEEE Trans. Parallel Distrib. Syst. 25 (3) (2014) 682–694, http://dx.doi.org/10.1109/TPDS.2013.57.

[9] H.R. Faragardi, M.R. Saleh Sedghpour, S. Fazliahmadi, T. Fahringer, N. Rasouli, GRP-HEFT: A budget-constrained resource provisioning scheme for workflow scheduling in IaaS clouds, IEEE Trans. Parallel Distrib. Syst. 31 (6) (2020) 1239–1254, http://dx.doi.org/10.1109/TPDS.2019.2961098.

[10] J.J. Durillo, R. Prodan, Multi-objective workflow scheduling in Amazon EC2, Clust. Comput. 17 (2) (2014) 169–189, http://dx.doi.org/10.1007/s10586-013-0325-0.

[11] I. Ahmad, Y.-K. Kwok, On exploiting task duplication in parallel program scheduling, IEEE Trans. Parallel Distrib. Syst. 9 (9) (1998) 872–892, http://dx.doi.org/10.1109/71.722221.

[12] R. Bajaj, D. Agrawal, Improving scheduling of tasks in a heterogeneous environment, IEEE Trans. Parallel Distrib. Syst. 15 (2) (2004) 107–118, http://dx.doi.org/10.1109/TPDS.2004.1264795.

[13] A. Gerasoulis, T. Yang, A comparison of clustering heuristics for scheduling directed acyclic graphs on multiprocessors, J. Parallel Distrib. Comput. 16 (4) (1992) 276–291, http://dx.doi.org/10.1016/0743-7315(92)90012-C.

[14] C. Boeres, J. Filho, V. Rebello, A cluster-based strategy for scheduling task on heterogeneous processors, in: 16th Symposium on Computer Architecture and High Performance Computing, 2004, pp. 214–221, http://dx.doi.org/10.1109/SBAC-PAD.2004.1.

[15] H. Kanemitsu, M. Hanada, H. Nakazato, Clustering-based task scheduling in a large number of heterogeneous processors, IEEE Trans. Parallel Distrib. Syst. 27 (11) (2016) 3144–3157, http://dx.doi.org/10.1109/TPDS.2016.2526682.

[16] K. Hu, G. Zeng, S. Ding, H. Jiang, Cluster-scheduling big graph traversal task for parallel processing in heterogeneous cloud based on DAG transformation, IEEE Access 7 (2019) 77070–77082, http://dx.doi.org/10.1109/ACCESS.2019.2921477.

[17] O. Sinnen, L. Sousa, Experimental evaluation of task scheduling accuracy: Implications for the scheduling model, IEICE Trans. Inf. Syst. 86 (9) (2003) 1620–1627.

[18] O. Sinnen, L. Sousa, Communication contention in task scheduling, IEEE Trans. Parallel Distrib. Syst. 16 (6) (2005) 503–515, http://dx.doi.org/10.1109/TPDS.2005.64.

[19] X. Tang, K. Li, D. Padua, Communication contention in APN list scheduling algorithm, Sci. China Ser. F: Inf. Sci. 52 (1) (2009) 59–69, http://dx.doi.org/10.1007/s11432-009-0010-3.

[20] O. Sinnen, A. To, M. Kaur, Contention-aware scheduling with task duplication, J. Parallel Distrib. Comput. 71 (1) (2011) 77–86, http://dx.doi.org/10.1016/j.jpdc.2010.10.004.

[21] P. Choudhury, P.P. Chakrabarti, R. Kumar, Online scheduling of dynamic task graphs with communication and contention for multiprocessors, IEEE Trans. Parallel Distrib. Syst. 23 (1) (2012) 126–133, http://dx.doi.org/10.1109/TPDS.2011.104.

[22] S.K. Roy, R. Devaraj, A. Sarkar, K. Maji, S. Sinha, Contention-aware optimal scheduling of real-time precedence-constrained task graphs on heterogeneous distributed systems, J. Syst. Archit. 105 (2020) 101706, http://dx.doi.org/10.1016/j.sysarc.2019.101706.

[23] S.K. Roy, R. Devaraj, A. Sarkar, SAFLA: Scheduling multiple real-time periodic task graphs on heterogeneous systems, IEEE Trans. Comput. 72 (4) (2023) 1067–1080, http://dx.doi.org/10.1109/TC.2022.3191970.

[24] Q. Zhang, Q. Wu, M. Zhou, J. Wen, S. Yao, A communication contention-cognizant scheduling approach for workflow execution across public and private clouds, IEEE Trans. Autom. Sci. Eng. 21 (4) (2024) 7040–7051, http://dx.doi.org/10.1109/TASE.2023.3336807.

[25] X. Shu, Q. Wu, M. Zhou, J. Wen, A communication-contention-aware privacy-preserving workflow scheduling method for geo-distributed datacenters, IEEE Trans. Serv. Comput. 17 (5) (2024) 1887–1898, http://dx.doi.org/10.1109/TSC.2024.3407595.

[26] C. Bolchini, A. Miele, Reliability-driven system-level synthesis for mixed-critical embedded systems, IEEE Trans. Comput. 62 (12) (2013) 2489–2502, http://dx.doi.org/10.1109/TC.2012.226.

[27] G. Juve, A. Chervenak, E. Deelman, S. Bharathi, G. Mehta, K. Vahi, Characterizing and profiling scientific workflows, Future Gener. Comput. Syst. 29 (3) (2013) 682–692, http://dx.doi.org/10.1016/j.future.2012.08.015.

[28] M. Rynge, K. Vahi, M.Z. Alam, E. Deelman, T. Miller, M. Livny, S. Knuth, J. Griffioen, J. Goodhue, D. Hudak, J. Ma, A. Pasquale, L. Fein, ACCESS pegasus: Bringing workflows to the ACCESS masses, in: Practice and Experience in Advanced Research Computing 2023: Computing for the Common Good, PEARC '23, ACM, New York, USA, 2023, pp. 478–480, http://dx.doi.org/10.1145/3569951.3597590.